

编译原理与设计 作业 5

07112303

左逸龙

1120231863

2026-06-30

Problem 1: 设有 SDD 如下:

产生式 语义规则

$S \rightarrow B\#$ $S.val = B.val$

$B \rightarrow B_1 0$ $B.val = 2 * B_1.val$

$B \rightarrow B_1 1$ $B.val = 2 * B_1.val + 1$

$B \rightarrow 1$ $B.val = 1$

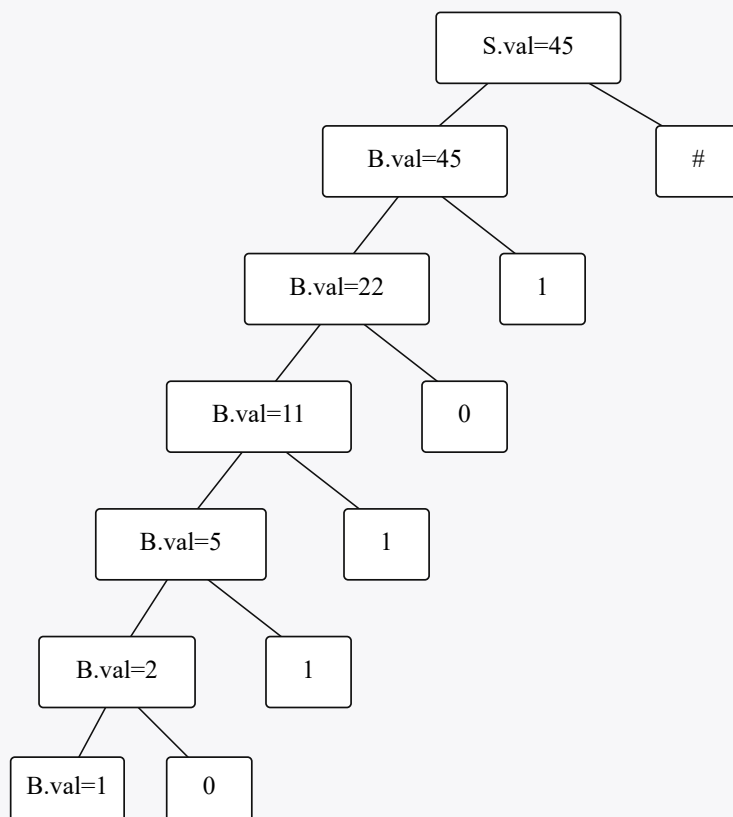
给出 101101 的注释语法分析树。

对串 101101# 的推导为:

$S \rightarrow B\#$

$B \rightarrow B_1 \rightarrow B_1 0 \rightarrow B_1 0 1 \rightarrow B_1 0 1 1 \rightarrow B_1 0 1 1 0 \rightarrow B_1 0 1 1 0 1 \rightarrow 1 0 1 1 0 1$

注释语法分析树如下:



Problem 2: 含小数点的二进制数文法定义如下:

$S \rightarrow L.L \mid L$

$$L \rightarrow LB \mid B$$
$$B \rightarrow 0 \mid 1$$

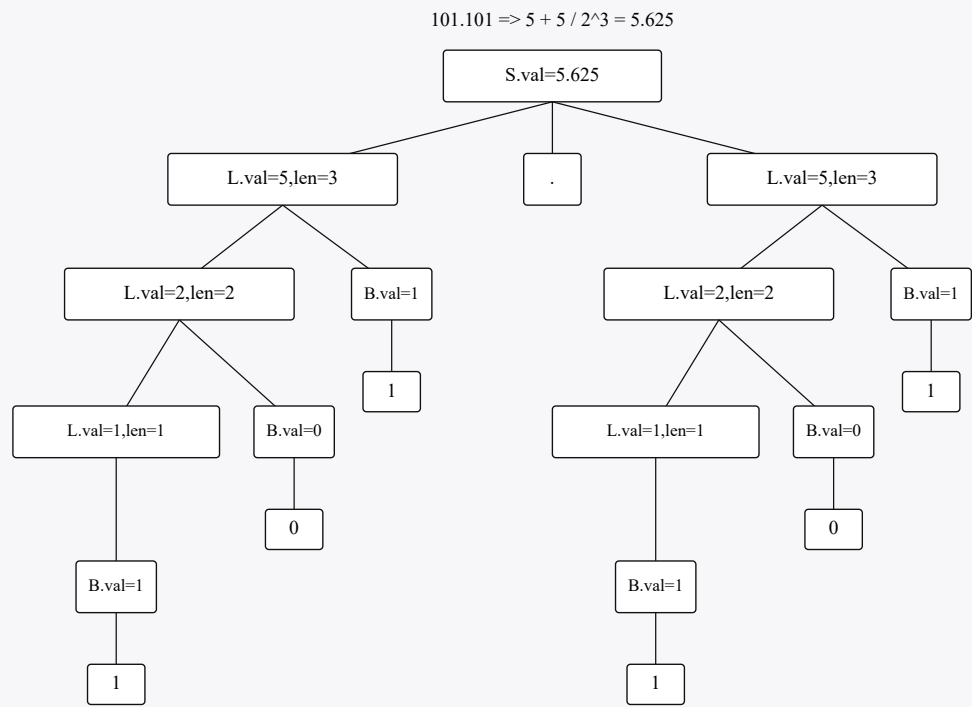
2.1 根据上述文法设计一个 SDD，实现二进制数到十进制数的翻译，比如 101.101 翻译为 5.625。

可为 L 设置两个综合属性： $L.val$ 表示当前二进制串按整数部分计算的值， $L.len$ 表示位数。
SDD 可写为：

产生式	语义规则
$S \rightarrow L_1.L_2$	$S.val = L1.val + L2.val / 2^{L2.len}$
$S \rightarrow L$	$S.val = L.val$
$L \rightarrow L_1B$	$L.val = 2 * L1.val + B.val; L.len = L1.len + 1$
$L \rightarrow B$	$L.val = B.val; L.len = 1$
$B \rightarrow 0$	$B.val = 0$
$B \rightarrow 1$	$B.val = 1$

2.2 给出 101.101 翻译为 5.625 的注释语法分析树。

注释语法分析树如下：



Problem 3: (选做) 在保证语义相同的情况下，改写 1 题中的 SDD，使文法不是左递归的。

改写后的文法为：

$$S \rightarrow B\#$$
$$B \rightarrow 1R$$

$$R \rightarrow 0R_1 \mid 1R_1 \mid \varepsilon$$

其中 R 使用继承属性 $R.in$ 记录已经读到的前缀值，综合属性 $R.val$ 返回最终结果。

对应 SDD 为：

产生式	语义规则
$S \rightarrow B\#$	$S.val = B.val$
$B \rightarrow 1R$	$R.in = 1; B.val = R.val$
$R \rightarrow 0R_1$	$R1.in = 2 * R.in; R.val = R1.val$
$R \rightarrow 1R_1$	$R1.in = 2 * R.in + 1; R.val = R1.val$
$R \rightarrow \varepsilon$	$R.val = R.in$

Problem 4: 设包含有数组引用的复制表达式的 SDD 如下：

产生式	语义规则
$S \rightarrow i = E;$	$gen(= E.place - i)$
$S \rightarrow L = E;$	$gen([]= E.place - L.base[L.addr])$
$E \rightarrow E_1 + E_2$	$E.place = newtemp(); gen(+ E1.place E2.place E.place)$
$E \rightarrow i$	$E.place = i$
$E \rightarrow L$	$E.place = newtemp(); gen([= L.base[L.addr] - E.place)$
$L \rightarrow i[E]$	$L.base = i; L.addr = newtemp(); gen(* E.place L.elem.width L.addr)$
$L \rightarrow L_1[E]$	$L.base = L1.base; L.type = L1.type; t = newtemp(); L.addr = newtemp(); gen(* E.place L.elem.width t); gen(+ L1.addr t L.addr)$

说明：

- $gen()$ 是产生四元式代码的函数，参数依次为操作符、第一操作数、第二操作数、计算结果；
- $newtemp()$ 是临时变量申请函数；
- 关于数组中用到的属性 $width$ 的 SDD 如下：

产生式	语义规则
$D \rightarrow BiC$	$w = B.width; i.width = C.width$
$B \rightarrow \text{int}$	$B.width = 4$
$B \rightarrow \text{float}$	$B.width = 8$
$C \rightarrow \varepsilon$	$C.width = w$
$C \rightarrow [\text{num}]C_1$	$C.width = \text{num.value} * C1.width; C.elem.width = C1.width$

- 语句中的数组说明： $\text{int } a[10][20], b[10][20]; c[10]$ 。

给出下列赋值语句的四元式代码：

4.1 $x = a[i][j] + b[i][j]$

数组 a, b 为 $\text{int}[10][20]$ ，所以第一维元素宽度为 80，第二维元素宽度为 4。

四元式为：

```
1 (*, i, 80, t1)
2 (*, j, 4, t2)
3 (+, t1, t2, t3)
4 ([=, a[t3], -, t4)
```

```

5 (*, i, 80, t5)
6 (*, j, 4, t6)
7 (+, t5, t6, t7)
8 ([=, b[t7], -, t8)
9 (+, t4, t8, t9)
10 ([=, t9, -, x)

```

4.2 a[b[i][j]][c[k]]=a[i][j]

先求右部 a[i][j] 的值，再求左部数组元素地址。

四元式为：

```

1 (*, i, 80, t1)
2 (*, j, 4, t2)
3 (+, t1, t2, t3)
4 ([=, a[t3], -, t4)
5 (*, i, 80, t5)
6 (*, j, 4, t6)
7 (+, t5, t6, t7)
8 ([=, b[t7], -, t8)
9 (*, k, 4, t9)
10 ([=, c[t9], -, t10)
11 (*, t8, 80, t11)
12 (*, t10, 4, t12)
13 (+, t11, t12, t13)
14 ([=, t4, -, a[t13])

```

Problem 5: 设有布尔表达式以及条件语句的 SDD 如下：

产生式	语义规则
$B \rightarrow E$	$B.true = makelist(nextinstr); B.false = makelist(nextinstr+1); gen(J_T\ E.PLACE - 0); gen(J_F\ E.PLACE - 0)$
$B \rightarrow E_1 == E_2$	$B.PLACE = NEWTEMP; gen(==\ E1.PLACE\ E2.PLACE\ B.PLACE); B.true = makelist(nextinstr); B.false = makelist(nextinstr+1); gen(J_T\ B.PLACE - 0); gen(J_F\ B.PLACE - 0)$
$B \rightarrow !B_1$	$B.true = B1.false; B.false = B1.true$
$B \rightarrow (B_1)$	$B.true = B1.true; B.false = B1.false$
$B \rightarrow B_1 \parallel MB_2$	$backpatch(B1.false, M.instr); B.true = merge(B1.true, B2.true); B.false = B2.false$
$B \rightarrow B_1 MB_2$	$backpatch(B1.true, M.instr); B.true = B2.true; B.false = merge(B1.false, B2.false)$
$S \rightarrow if (B) M_1 S_1 N else M_2 S_2$	$backpatch(B.true, M1.instr); backpatch(B.false, M2.instr); S.next = merge(S1.next, N.next, S2.next)$
$S \rightarrow while\ M_1(B) M_2 S_1$	$backpatch(S1.next, M1.instr); backpatch(B.true, M2.instr); S.next = B.false; gen(J - - M1.instr)$
$M \rightarrow \epsilon$	$M.instr = nextinstr$
$N \rightarrow \epsilon$	$N.next = makelist(nextinstr); gen(J - - 0)$
$S \rightarrow \{L\}$	$S.next = L.next$
$S \rightarrow i = E;$	$S.next = null$
$L \rightarrow L_1 MS$	$backpatch(L1.next, M.instr); L.next = S.next;$
$L \rightarrow S$	$L.next = S.next$

SDD 的说明：

a) 四元式编号作为跳转地址使用。

b) 函数 `makelist(x)` 表示出现了标号的引用时不知道地址的情况，函数参数 `x` 为标号的使用位置，单遍处理生成中间代码需要使用拉链-返填技术，假设通过标号表记录相关信息，此处为标号的引用出现，按标号引用记录入标号表。

c) 函数 `backpatch(x,y)` 表示标号 `x` 的定义位置在 `y`，需要返填标号 `x` 的使用位置。

d) 函数 `merge(x,y,z,...)` 表示标号 `x,y,z,...` 合并为一个标号。

e) `nextinstr` 表示下一条四元式的编号。

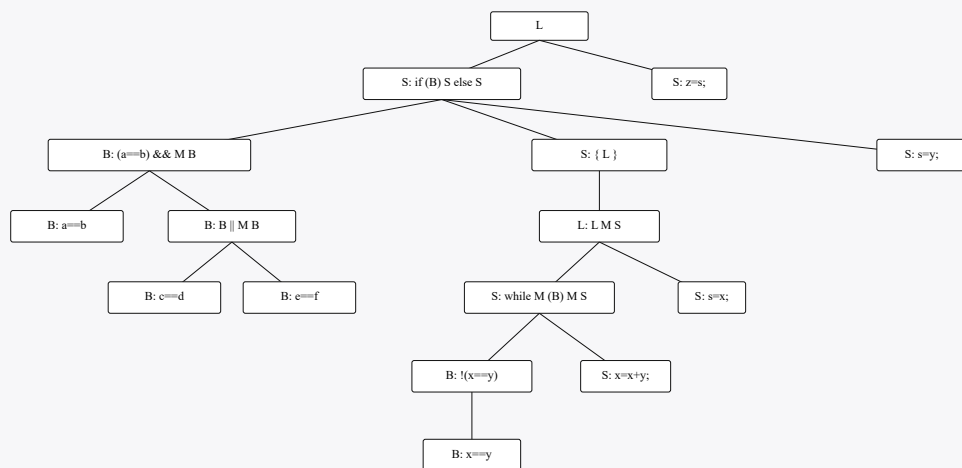
有程序段：

```
1 if (a==b&&(c==d||e==f))
2   {while(!(x==y))
3     x=x+y;
4     s=x;}
5 else
6   s=y;
7 z=s;
```

问题：

5.1 给出该程序段的语法分析树（表达式 E 结构使用 4 题的）。

语法分析树如下：



5.2 给出该程序段语义处理后的四元式形式的目标代码（代码中的表达式 E 结构使用 4 题的 SDD）。

四元式编号从 1 开始：

```

1 (1)  (==, a, b, t1)
2 (2)  (J_T, t1, -, 4)
3 (3)  (J_F, t1, -, 18)
4 (4)  (==, c, d, t2)
5 (5)  (J_T, t2, -, 10)
6 (6)  (J_F, t2, -, 7)
7 (7)  (==, e, f, t3)
8 (8)  (J_T, t3, -, 10)
9 (9)  (J_F, t3, -, 18)
10 (10) (==, x, y, t4)
11 (11) (J_T, t4, -, 16)
12 (12) (J_F, t4, -, 13)
13 (13) (+, x, y, t5)
14 (14) (=, t5, -, x)
15 (15) (J, -, -, 10)
16 (16) (=, x, -, s)
17 (17) (J, -, -, 19)
18 (18) (=, y, -, s)
19 (19) (=, s, -, z)

```

5.3 给出该程序段语义处理后的拉链-返填使用的标号表。

标号表按以下示例内容填写：

标号名	定义否 (1/0)	返填顺序
L_i	1	⑤→②→①
...		

标号名	定义否 (1/0)	返填顺序
$L_1 = 4$	1	②
$L_2 = 18$	1	③→⑨
$L_3 = 7$	1	⑥
$L_4 = 10$	1	⑤→⑧
$L_5 = 16$	1	⑪
$L_6 = 13$	1	⑫
$L_7 = 19$	1	⑰